SOLICODE VEILLE TECHNOLOGIQUE

Modularité et Fonctions en PHP

Structurer, réutiliser et pérenniser son code d'application Web

Format Oral
3 Minutes

1 Technologie & Modularité

PHP (Hypertext Preprocessor)
Langage de programmation dynamique **côté serveur**, conçu pour traiter les requêtes HTTP, interagir avec les bases de données et générer des pages Web dynamiques.

Définition de la modularité
Approche d'architecture consistant à **diviser le code en plusieurs fichiers et fonctions réutilisables** pour rendre une application plus organisée, évolutive et facile à maintenir.

2 Problème résolu

Sans modularité (code monolithique)
Code dupliqué manuellement (copier-coller), fichiers géants illisibles, incohérences lors des mises à jour et risque d'erreurs en cascade.

- ✓ **Zéro duplication :** évite la répétition du même code dans plusieurs pages Web.
- ✓ **Réduction des erreurs :** élimine les incohérences et fiabilise les traitements.
- ✓ **Modifications simplifiées :** une mise à jour dans un fichier s'applique partout immédiatement.

3 Les trois mots-clés fondamentaux

`require_once` **Inclusion sécurisée unique**
Permet d'inclure un fichier PHP externe **une seule fois** au cours du script, évitant ainsi les erreurs fatales de redéclaration.

`Fonctions` **Blocs logiques réutilisables**
Permettent d'exécuter plusieurs fois un traitement logique (calcul, affichage, validation) en l'appelant avec des paramètres, **sans répéter le code**.

`DRY` **Don't Repeat Yourself**
Principe d'ingénierie logicielle stipulant que chaque logique métier doit avoir **une représentation unique et non ambiguë** dans le système.

4 Avantages majeurs

Code propre & organisé
Architecture claire, découpage par responsabilités et lisibilité optimale.

Réduction des erreurs
Une seule source de vérité testée, minimisant les bugs et effets de bord.

Maintenance facile
Correction ou ajout de fonctionnalités centralisé sans impacter le reste.

Gain de temps
Réutilisation immédiate des fonctions éprouvées et accélération du projet.

Règle d'or : « Écrire moins de code pour créer plus de valeur »